

Homework 03: Solutions

Question 1: Build an explain-to-me skill

Appended the complete contents of the developed `explain-to-me/SKILL.md` file at the end of this Homework 3 submission under the heading `explain-to-me skill`.

Question 2: Ridge regression with nearly collinear predictors

Solution 2:

```
set.seed(43231)

n <- 100
p <- 4

## generate the five independent standard normal sources, in the order given
L <- rnorm(n)
U1 <- rnorm(n)
U2 <- rnorm(n)
X3 <- rnorm(n)
X4 <- rnorm(n)

X1 <- L + 0.04 * U1
X2 <- L + 0.04 * U2

Xraw <- cbind(X1, X2, X3, X4)

standardize <- function(x) {
  xc <- x - mean(x)
  xc / sqrt(mean(xc^2)) # mean 0, average squared value 1
}

X <- apply(Xraw, 2, standardize)
colnames(X) <- c("X1", "X2", "X3", "X4")
```

Q2(a) Correlation, singular values, and why the contrast is unstable:

```
cor12 <- cor(X[, 1], X[, 2])
sv     <- svd(X)$d

cat("corr(X1, X2)      :", round(cor12, 4), "\n")
```

```
corr(X1, X2)      : 0.9977
```

```
cat("singular values  :", round(sv, 4), "\n")
```

```
singular values   : 14.1415 10.3477 9.629 0.4754
```

```
cat("largest sv       :", round(max(sv), 4), "\n")
```

```
largest sv        : 14.1415
```

```
cat("smallest sv      :", round(min(sv), 4), "\n")
```

```
smallest sv       : 0.4754
```

X_1, X_2 both come from L plus tiny independent noise (0.04), so they're nearly identical; $\text{corr} \approx 0.998$. Singular values 14.14, 10.35, 9.63, 0.48 show it: the last one drops off a cliff since $X_1 - X_2$ barely moves X . Since $\text{Var}(\hat{\beta}) = \sigma^2 \mathbf{V} \mathbf{D}^{-2} \mathbf{V}^T$, small singular values blow up variance. $(1, -1, 0, 0)/\sqrt{2}$ sits on $d_{\min}$, so $\hat{\beta}_1 - \hat{\beta}_2$ is unstable; $(1, 1, 0, 0)/\sqrt{2}$ sits on the large values, so $\hat{\beta}_1 + \hat{\beta}_2$ stays stable. With $d_{\min} \approx 0.48$, $d_{\max} \approx 14.14$, that gap squared is huge.

Q2(b) 200 repetitions across $\lambda \in \{0, 0.02, 0.2\}$:

```
beta_true <- c(1.5, 1.5, 1, 0)
mu <- as.numeric(X %*% beta_true)

lambdas <- c(0, 0.02, 0.2)
Rreps <- 200
XtX <- t(X) %*% X

store <- lapply(lambdas, function(lam) {
  list(b1 = numeric(Rreps), b2 = numeric(Rreps),
       train_mse = numeric(Rreps), test_mse = numeric(Rreps))
})
names(store) <- as.character(lambdas)
```

```

for (r in seq_len(Rreps)) {
  eps  <- rnorm(n)
  epss <- rnorm(n)
  y    <- mu + eps
  ys   <- mu + epss

  ybar  <- mean(y)
  ytilde <- y - ybar

  for (lam in lambdas) {
    A      <- XtX + n * lam * diag(p)
    beta_hat <- solve(A, t(X) %*% ytilde)
    beta0   <- ybar

    fitted   <- as.numeric(beta0 + X %*% beta_hat)
    train_mse <- mean((y - fitted)^2)
    test_mse  <- mean((ys - fitted)^2)

    key <- as.character(lam)
    store[[key]]$b1[r] <- beta_hat[1]
    store[[key]]$b2[r] <- beta_hat[2]
    store[[key]]$train_mse[r] <- train_mse
    store[[key]]$test_mse[r] <- test_mse
  }
}

summary_tbl <- do.call(rbind, lapply(lambdas, function(lam) {
  key <- as.character(lam)
  b1  <- store[[key]]$b1
  b2  <- store[[key]]$b2
  s    <- b1 + b2
  d    <- b1 - b2
  data.frame(
    lambda      = lam,
    mean_beta1  = mean(b1), sd_beta1  = sd(b1),
    mean_beta2  = mean(b2), sd_beta2  = sd(b2),
    mean_sum    = mean(s),  sd_sum    = sd(s),
    mean_diff   = mean(d),  sd_diff   = sd(d),
    mean_train_mse = mean(store[[key]]$train_mse),
    mean_test_mse  = mean(store[[key]]$test_mse)
  )
}))

knitr::kable(summary_tbl, digits = 4)

```

lambda	mean_beta1	sd_beta1	mean_beta2	sd_beta2	mean_sum	sd_sum	mean_diff	sd_diff	mean_train_mse	mean_test_mse
0.00	1.4956	1.5439	1.5098	1.5486	3.0054	0.0951	-	3.0911	0.9648	1.0625
							0.0142			
0.02	1.4870	0.1615	1.4892	0.1659	2.9762	0.0942	-	0.3136	0.9747	1.0493
							0.0022			
0.20	1.3679	0.0457	1.3687	0.0473	2.7366	0.0865	-	0.0344	1.0768	1.1335
							0.0008			

Q2c Solution:

As λ grows, $\hat{\beta}_1, \hat{\beta}_2$ drift from 1.5 (a little bias) but variance drops fast. `sd_diff` falls from ~ 3 to near 0, since ridge's $n\lambda$ term hits the tiny singular value hardest, exactly the one driving $\hat{\beta}_1 - \hat{\beta}_2$. `sd_sum` barely changes. Test MSE dips at $\lambda = 0.02$, then rises at $\lambda = 0.2$ once bias dominates: classic bias variance trade-off.

Fitted values move much less than the coefficients, since $X\hat{\beta}$ only depends on well-determined directions; the unstable $X_1 - X_2$ direction barely affects $X\hat{\beta}$ to begin with.

Question 3

Q3(a) The ridge normal equation and its uniqueness (Solution):

The ridge objective is

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2.$$

Its gradient is

$$\nabla L_\lambda(\beta) = -\frac{1}{n} \mathbf{X}^\top (\tilde{\mathbf{y}} - \mathbf{X}\beta) + \lambda\beta.$$

Setting this to $\mathbf{0}$ and multiplying through by n gives

$$-\mathbf{X}^\top \tilde{\mathbf{y}} + \mathbf{X}^\top \mathbf{X}\beta + n\lambda\beta = \mathbf{0} \implies (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \hat{\beta}_\lambda = \mathbf{X}^\top \tilde{\mathbf{y}}.$$

$\mathbf{X}^\top \mathbf{X}$ is symmetric positive **semidefinite**: for any $\mathbf{v} \neq \mathbf{0}$, $\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0$, with equality possible whenever $\mathbf{X}$ is rank deficient or $p > n$ (so $\mathbf{X}^\top \mathbf{X}$ can have zero eigenvalues). Adding $n\lambda \mathbf{I}_p$ shifts every eigenvalue up by $n\lambda > 0$, so all eigenvalues of $\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p$ are strictly positive — the matrix is positive definite and hence invertible, regardless of the rank of $\mathbf{X}$. The normal equation therefore has the unique solution $\hat{\beta}_\lambda = (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \tilde{\mathbf{y}}$ whenever $\lambda > 0$.

Q3(b) Shrinkage in the singular-value directions (Solution):

Write the compact SVD $\mathbf{X} = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^\top$, so $\mathbf{U}_r^\top \mathbf{U}_r = \mathbf{V}_r^\top \mathbf{V}_r = \mathbf{I}_r$ and $\mathbf{X}^\top \mathbf{X} = \mathbf{V}_r \mathbf{D}_r^2 \mathbf{V}_r^\top$. Any component of β orthogonal to $\text{col}(\mathbf{V}_r)$ adds to the penalty $\|\beta\|_2^2$ without affecting the fit, so the

minimizer lies in $\text{col}(\mathbf{V}_r)$: write $\hat{\beta}_\lambda = \mathbf{V}_r \alpha$. Substituting into the normal equation and left-multiplying by $\mathbf{V}_r^\top$ (using $\mathbf{V}_r^\top \mathbf{V}_r = \mathbf{I}_r$) gives

$$(\mathbf{D}_r^2 + n\lambda \mathbf{I}_r) \alpha = \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}} \implies \alpha_j = \frac{d_j (\mathbf{u}_j^\top \tilde{\mathbf{y}})}{d_j^2 + n\lambda}, \quad j = 1, \dots, r.$$

The fitted values are $\mathbf{X} \hat{\beta}_\lambda = \mathbf{U}_r \mathbf{D}_r \alpha$, so the component of the fit along $\mathbf{u}_j$ is

$$d_j \alpha_j = \frac{d_j^2}{d_j^2 + n\lambda} (\mathbf{u}_j^\top \tilde{\mathbf{y}}) = \rho_j(\lambda) (\mathbf{u}_j^\top \tilde{\mathbf{y}}), \quad \rho_j(\lambda) = \frac{d_j^2}{d_j^2 + n\lambda}.$$

Since $\mathbf{u}_j^\top \tilde{\mathbf{y}}$ is exactly the OLS fitted component along $\mathbf{u}_j$ (the $\lambda = 0$ case gives $\rho_j(0) = 1$), ridge multiplies each left-singular-vector component of the fit by the shrinkage factor $\rho_j(\lambda)$.

```
d <- c(10, 2, 0.2)
n3 <- 100
lam3 <- 0.01

rho <- d^2 / (d^2 + n3 * lam3)
df_eff <- 1 + sum(rho)

data.frame(j = 1:3, d_j = d, rho_j = round(rho, 4)) |> knitr::kable()
```

j	d_j	rho_j
1	10.0	0.9901
2	2.0	0.8000
3	0.2	0.0385

```
cat("df_eff(lambda) =", round(df_eff, 4), "\n")
```

```
df_eff(lambda) = 2.8286
```

So $\rho_1 = 100/101 \approx 0.9901$, $\rho_2 = 4/5 = 0.8$, $\rho_3 = 0.04/1.04 \approx 0.0385$, and $\text{df}_{\text{eff}}(\lambda) = 1 + \rho_1 + \rho_2 + \rho_3 \approx 2.8286$.

Q3(c) Which direction shrinks most, and why (Solution):

$d_3 = 0.2$ shrinks hardest ($\rho_3 \approx 0.038$): its OLS component is mostly noise, so shrinking it helps. But if the true β actually has weight there, ridge kills real signal too, biasing the fit.

Question 4

Q4(a) Condition numbers

```

ev0 <- c(9, 0.01)
lam4 <- 0.09
evA <- ev0 + lam4

cond0 <- max(ev0) / min(ev0)
condA <- max(evA) / min(evA)

cat("eigenvalues of A0      :", ev0, "   cond(A0)      =", cond0, "\n")

```

```
eigenvalues of A0      : 9 0.01   cond(A0)      = 900
```

```
cat("eigenvalues of A_lambda:", evA, "   cond(A_lambda) =", round(condA, 3), "\n")
```

```
eigenvalues of A_lambda: 9.09 0.1   cond(A_lambda) = 90.9
```

$\mathbf{A}_0$ has condition number $9/0.01 = 900$: the objective's contours are extremely elongated ellipses, almost flat along the small-eigenvalue direction. Adding $\lambda \mathbf{I}_p$ shifts both eigenvalues up by the same amount (9.09 and 0.1), so the condition number drops to $9.09/0.1 = 90.9$ — about a 10-fold improvement. Geometrically, the ridge penalty makes the level sets of the objective rounder (less eccentric), since it raises the curvature disproportionately in the previously-flat direction.

Q4(b) Gradient descent contraction

L_λ is quadratic in β with curvature $\mathbf{A}_\lambda$, so it can be written $L_\lambda(\beta) = \frac{1}{2}(\beta - \hat{\beta}_\lambda)^\top \mathbf{A}_\lambda (\beta - \hat{\beta}_\lambda) + \text{const}$, so $\nabla L_\lambda(\beta) = \mathbf{A}_\lambda(\beta - \hat{\beta}_\lambda)$. Subtracting $\hat{\beta}_\lambda$ from both sides of the update rule,

$$\mathbf{e}^{(k+1)} = \beta^{(k+1)} - \hat{\beta}_\lambda = \mathbf{e}^{(k)} - \eta \mathbf{A}_\lambda \mathbf{e}^{(k)} = (\mathbf{I} - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

If $\mathbf{v}$ is an eigenvector of $\mathbf{A}_0$ with eigenvalue a , it is also an eigenvector of $\mathbf{A}_\lambda = \mathbf{A}_0 + \lambda \mathbf{I}_p$ with eigenvalue $a + \lambda$, and hence of $\mathbf{I} - \eta \mathbf{A}_\lambda$ with eigenvalue $1 - \eta(a + \lambda)$. So the component of $\mathbf{e}^{(k)}$ along that eigendirection is multiplied by $1 - \eta(a + \lambda)$ at every step, as required.

```

M <- max(evA)           # 9.09

interval_upper <- 2 / M
cat("contracts every eigendirection for 0 <", "eta <", round(interval_upper, 4), "\n\n")

```

```
contracts every eigendirection for 0 < eta < 0.22
```

```

eta1 <- 1 / M
factors_eta1 <- 1 - eta1 * evA
cat("eta = 1/M   =", round(eta1, 4), " factors:", round(factors_eta1, 4), "\n")

```

```
eta = 1/M   = 0.11  factors: 0 0.989
```

```
eta2 <- 2.1 / M
factors_eta2 <- 1 - eta2 * evA
cat("eta = 2.1/M =", round(eta2, 4), " factors:", round(factors_eta2, 4), "\n")
```

```
eta = 2.1/M = 0.231  factors: -1.1 0.9769
```

Contraction ($|1 - \eta(a + \lambda)| < 1$) for both eigenvalues requires $0 < \eta < 2/M$ where $M = 9.09$ is the larger eigenvalue of $\mathbf{A}_\lambda$ (this is the binding constraint since it's the larger of the two). Numerically that's $0 < \eta < 0.2200$.

- At $\eta = 1/M \approx 0.11$: the factor along the large-curvature direction ($a + \lambda = 9.09$) is $1 - 1 = 0$ — that component is annihilated in a single step. The factor along the flat direction ($a + \lambda = 0.1$) is $1 - 0.11(0.1) \approx 0.989$ — barely contracts each step. So the high-curvature (well-determined) direction converges essentially immediately, while the flat direction converges very slowly; overall convergence speed is governed by the slow direction.
- At $\eta = 2.1/M \approx 0.231$: the factor along the large-curvature direction is $1 - 2.1 = -1.1$, with $|-1.1| > 1$. Since the initial error has a nonzero component along that eigendirection, that component grows in magnitude and alternates sign every step, so the iterates diverge — gradient descent does **not** converge at this step size, even though the other direction's factor (≈ 0.977) would contract on its own.

Q4(c) Why tuning λ for fast convergence is not valid

Tuning λ for fast convergence optimizes conditioning, not prediction, and since bigger λ always helps conditioning, it'd just push λ too far with no natural stopping point. Also kind of moot, since ridge already has a closed form, $\hat{\beta}_\lambda = (X^T X + n\lambda I)^{-1} X^T \tilde{y}$. λ should come from CV or GCV, not convergence speed.

Question 5: Selecting the ridge penalty for real-estate prediction

Solution:

```
realestate <- read.csv("data/realestate.csv")
folds      <- read.csv("data/realestate-split-folds.csv")

dat <- merge(realestate, folds, by = "row_id")

predictor_cols <- setdiff(names(realestate), c("row_id", "price"))

train <- dat[dat$split == "train", ]
test  <- dat[dat$split == "test", ]

lambda_grid <- c(0, 10^(-4 + 0.05 * (0:120)))
```

```

standardize_fit <- function(Xmat) {
  # centers columns and scales by sqrt(mean of squared centered values)
  # (variance divided by n, per the assignment's convention)
  mu <- colMeans(Xmat)
  Xc <- sweep(Xmat, 2, mu, "-")
  sc <- sqrt(colMeans(Xc^2))
  list(mu = mu, sc = sc)
}

standardize_apply <- function(Xmat, params) {
  Xc <- sweep(Xmat, 2, params$mu, "-")
  sweep(Xc, 2, params$sc, "/")
}

ridge_fit <- function(Xstd, y, lambda) {
  n <- nrow(Xstd); p <- ncol(Xstd)
  ybar <- mean(y)
  ytilde <- y - ybar
  A <- crossprod(Xstd) + n * lambda * diag(p)
  beta <- solve(A, crossprod(Xstd, ytilde))
  list(beta0 = ybar, beta = as.numeric(beta))
}

ridge_predict <- function(fit, Xstd) {
  as.numeric(fit$beta0 + Xstd %*% fit$beta)
}

effective_df <- function(Xstd, lambda) {
  # 1 (intercept) + sum of d_j^2 / (d_j^2 + n*lambda)
  n <- nrow(Xstd)
  d <- svd(Xstd)$d
  1 + sum(d^2 / (d^2 + n * lambda))
}

```

Q5(a) Ten-fold cross-validation

```

fold_ids <- sort(unique(train$cv_fold))
cv_mse <- matrix(NA, nrow = length(fold_ids), ncol = length(lambda_grid))

for (k in seq_along(fold_ids)) {
  val_idx <- train$cv_fold == fold_ids[k]
  fit_idx <- !val_idx

  Xfit <- as.matrix(train[fit_idx, predictor_cols])
  yfit <- train$price[fit_idx]
}

```

```

Xval <- as.matrix(train[val_idx, predictor_cols])
yval <- train$price[val_idx]

sp <- standardize_fit(Xfit)
Xfit_std <- standardize_apply(Xfit, sp)
Xval_std <- standardize_apply(Xval, sp)

for (l in seq_along(lambda_grid)) {
  fit <- ridge_fit(Xfit_std, yfit, lambda_grid[l])
  pred <- ridge_predict(fit, Xval_std)
  cv_mse[k, l] <- mean((yval - pred)^2)
}
}

mean_mse <- colMeans(cv_mse)
se_mse <- apply(cv_mse, 2, sd) / sqrt(length(fold_ids))

lambda_min_idx <- which.min(mean_mse)
lambda_min <- lambda_grid[lambda_min_idx]
thresh <- mean_mse[lambda_min_idx] + se_mse[lambda_min_idx]

# largest lambda (including OLS, lambda = 0) with mean MSE <= threshold
ok <- which(mean_mse <= thresh)
lambda_1se <- max(lambda_grid[ok])

plot(lambda_grid, mean_mse, log = "x", type = "l",
      xlab = expression(lambda), ylab = "mean CV MSE",
      main = "10-fold CV curve")

```

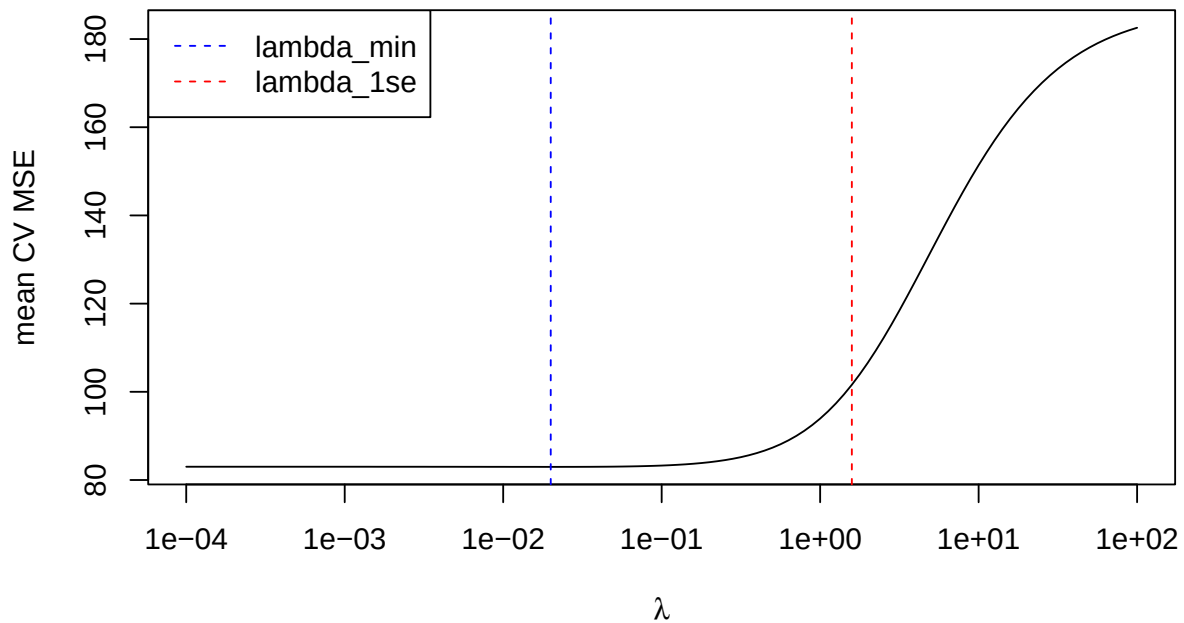
Warning in xy.coords(x, y, xlabel, ylabel, log): 1 x value <= 0 omitted from logarithmic plot

```

abline(v = lambda_min, col = "blue", lty = 2)
abline(v = lambda_1se, col = "red", lty = 2)
legend("topleft", legend = c("lambda_min", "lambda_1se"),
      col = c("blue", "red"), lty = 2)

```

10-fold CV curve



```
cat("lambda_min :", lambda_min, "\n")
```

```
lambda_min : 0.01995262
```

```
cat("lambda_1se :", lambda_1se, "\n")
```

```
lambda_1se : 1.584893
```

Q5(b) GCV on the full training set

```
Xtr <- as.matrix(train[, predictor_cols])
ytr <- train$price

sp_full <- standardize_fit(Xtr)
Xtr_std <- standardize_apply(Xtr, sp_full)

gcv <- numeric(length(lambda_grid))
for (l in seq_along(lambda_grid)) {
  fit <- ridge_fit(Xtr_std, ytr, lambda_grid[l])
  pred <- ridge_predict(fit, Xtr_std)
  train_mse <- mean((ytr - pred)^2)
```

```

df <- effective_df(Xtr_std, lambda_grid[1])
n_tr <- nrow(Xtr_std)
gcv[1] <- train_mse / (1 - df / n_tr)^2
}

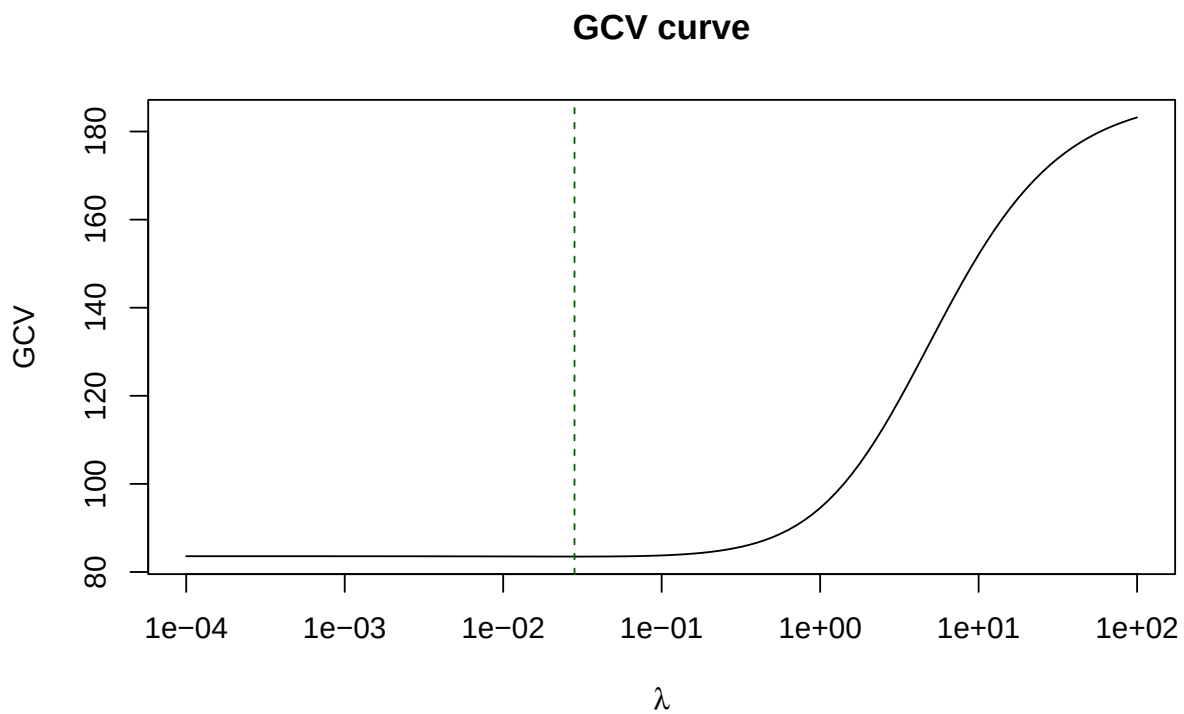
lambda_gcv <- lambda_grid[which.min(gcv)]

plot(lambda_grid, gcv, log = "x", type = "l",
      xlab = expression(lambda), ylab = "GCV",
      main = "GCV curve")

```

Warning in xy.coords(x, y, xlabel, ylabel, log): 1 x value <= 0 omitted from logarithmic plot

```
abline(v = lambda_gcv, col = "darkgreen", lty = 2)
```



```
cat("lambda_GCV :", lambda_gcv, "\n")
```

```
lambda_GCV : 0.02818383
```

Q5(c) Refit and compare on the test set

```
Xte <- as.matrix(test[, predictor_cols])
yte <- test$price
Xte_std <- standardize_apply(Xte, sp_full)

lambdas_final <- c(OLS = 0, lambda_min = lambda_min,
                  lambda_1se = lambda_1se, lambda_GCV = lambda_gcv)

final_tbl <- do.call(rbind, lapply(names(lambdas_final), function(nm) {
  lam <- lambdas_final[nm]
  fit <- ridge_fit(Xtr_std, ytr, lam)
  pred_tr <- ridge_predict(fit, Xtr_std)
  pred_te <- ridge_predict(fit, Xte_std)
  data.frame(
    model = nm, lambda = lam,
    train_mse = mean((ytr - pred_tr)^2),
    test_mse = mean((yte - pred_te)^2),
    df_eff = effective_df(Xtr_std, lam)
  )
}))

knitr::kable(final_tbl, digits = 4)
```

	model	lambda	train_mse	test_mse	df_eff
OLS	OLS	0.0000	80.0939	66.3225	7.0000
lambda_min	lambda_min	0.0200	80.1354	65.9996	6.7732
lambda_1se	lambda_1se	1.5849	100.3736	81.1998	3.0329
lambda_GCV	lambda_GCV	0.0282	80.1713	65.9028	6.6887

OLS wins on training MSE (no penalty). λ_{1se} picks a much simpler model (df_{eff} : $7 \rightarrow 3$), costing both train and test MSE (81.20 vs. ~ 66). λ_{min} , λ_{GCV} barely shrink anything.

λ_{GCV} edges out the rest here (65.90 vs. 66.00 vs. 66.32), but that's not meaningful; all three were tuned on training data alone before touching the test set. Picking whichever wins *on the test set* would just reward luck, not generalization.

explain-to-me skill

What the 'explain-to-me' skill gave as output:

Q2(a) — Correlation, singular values, and instability You're being asked to look at a design matrix where two of the four predictors are built to be almost identical (each is the same latent variable

plus a tiny bit of independent noise), and connect that construction to what happens numerically. Concretely: compute the correlation between those two predictors, get the singular values of the standardized design matrix, and then explain — using the variance formula for the least-squares/ridge estimator in terms of the SVD — why one particular linear combination of the two coefficients is poorly estimated while another combination is fine. The deliverable is a short written explanation tying the small singular value to instability in a specific contrast, not just reporting numbers.

Q2(b) — Simulation across values This asks you to run a repeated-simulation study: for several ridge penalty values, refit many times on fresh noise, and track how the individual coefficients (and their sum/difference) behave, along with train and test error. The deliverable is a summary table (means and standard deviations across repetitions) for each .

Q2(c) — Interpreting the simulation Using the table from (b), you're asked to describe the pattern you see as increases — how bias and variance in the unstable contrast trade off, and what happens to prediction error. This is a written interpretation, not new computation.

Relevant lecture concepts:

The link between singular values, coefficient variance, and shrinkage is the core of Ridge Regression: Stability Through Shrinkage — <https://teazrq.github.io/stat432rpy/topics/ridge-regression/ridge-regression.html> How enters the fitted solution and reshapes variance/bias is covered in From a Penalized Objective to a Fitted Ridge Model — <https://teazrq.github.io/stat432rpy/topics/ridge-regression/optimization-and-cross-validation.html>

Reference

The real-estate data come from I-Cheng Yeh's [Real Estate Valuation dataset](#) in the UCI Machine Learning Repository. The distributed data retain the original observations and add only the course train-test and fold assignments.

Listing 1 explain-to-me/SKILL.md

```
---
name: explain-to-me
description: Explains what a homework question is asking, in plain language, without providing
---

# explain-to-me

## Purpose
When I explicitly ask you to use this skill on a specific homework question, read
that question and explain it to me. Do not solve it.

For the question I point you to, explain:
1. What the question is fundamentally asking me to do, in plain conversational
   language (not a restatement of the notation).
2. What deliverables are actually required: what I need to derive, compute, plot,
   or report, and in what form.
3. Which concept(s) from the Week 3 ridge-regression lectures apply, and point me to
   the specific relevant section of:
   - Ridge Regression: Stability Through Shrinkage -
     https://teazrq.github.io/stat432rpy/topics/ridge-regression/ridge-regression.html
   - From a Penalized Objective to a Fitted Ridge Model -
     https://teazrq.github.io/stat432rpy/topics/ridge-regression/optimization-and-cross-validation.html

## Rules
- Only activate when I explicitly ask you to use "explain-to-me" on a named question
  or sub-part.
- Never write, sketch, or hint at the solution: no code, no derivation steps, no
  numeric answers, no "the answer will look like X."
- Do not solve any sub-part for me, even partially - explain what is being asked,
  not how to get there.
- Keep it short: a few plain-language sentences per sub-part is enough.
- If my follow-up question pushes toward the actual solution, decline and redirect
  me back to the lecture sections instead of answering it.

## Example invocation
"Use explain-to-me to read Question 2."
```
